

# Complexity Limiting

## GraphQL Complexity Limiting

Even with a simple schema, deeply nested or resource-heavy operations can overload your upstream services.

```
type Query {  
  continents: [Continent!]!  
}
```

```
type Continent {  
  name: String!  
  countries: [Country!]!  
}
```

```
type Country {  
  name: String!  
  continent: Continent!  
}
```

# Complexity Limiting

## Deeply Nested Query Example

```
{  
  continents {  
    countries {  
      continent {  
        countries {  
          continent {  
            countries {  
              name  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

This query is valid, but heavily recursive and can stress your backend.

# Complexity

## Limiting

Limit query depth by setting a max depth in:

- A Policy
- An Individual Key

Query Depth = 2

```
{  
  
  continents {  
  
    name  
  
  }  
  
}
```

Query Depth = 3

```
{  
  
  continents {  
  
    countries {  
  
      name  
  
    }  
  
  }  
  
}
```

**When Depth Limit is Exceeded:**

**Tyk will block the query and respond with:**

```
{  
  
  "error": "depth limit  
exceeded"  
  
}
```